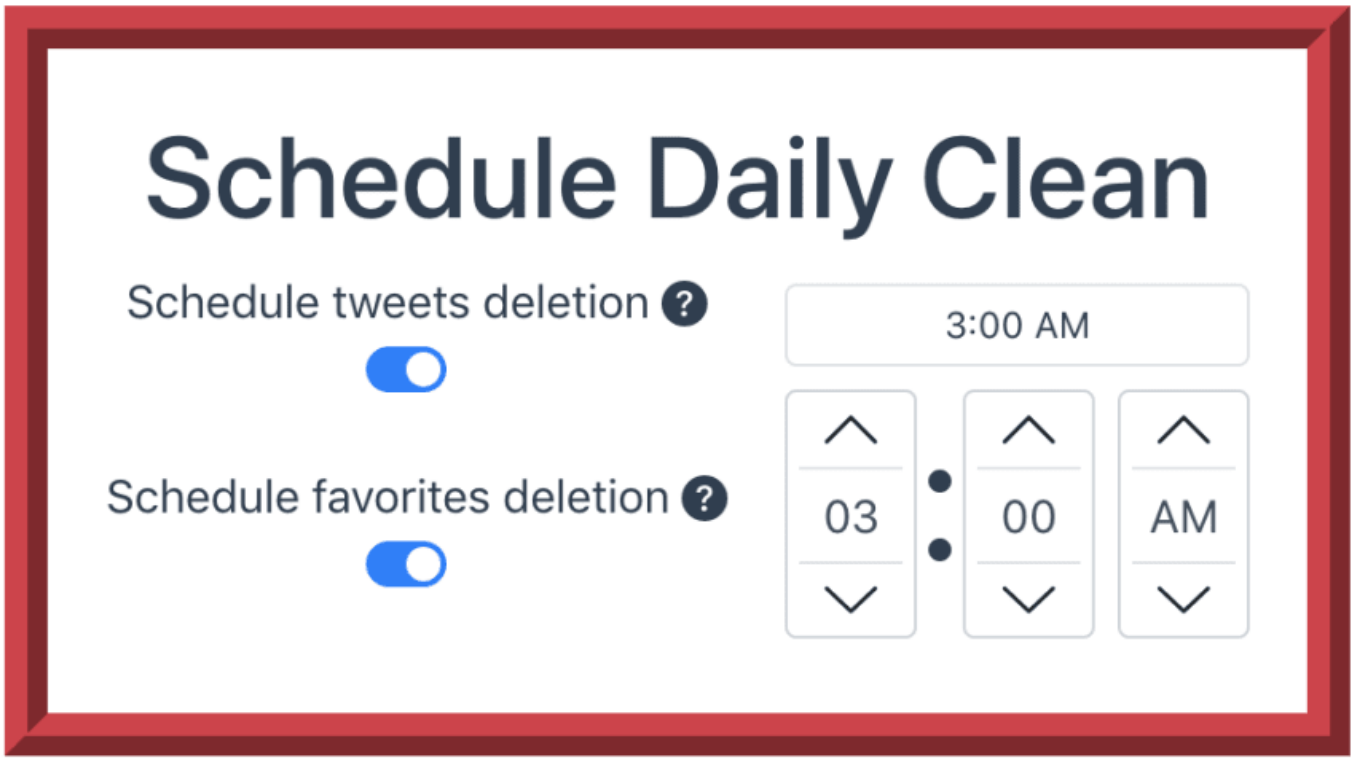
Nick Gottschlich
Posted on Aug 12, 2020

How to create a scheduler with with Electron, Vue and node-schedule

#electron #vue #node #javascript

Hello, my name is Nick Gottschlich, and I am the creator of Social Amnesia, an Electron/Vue application to help you delete your reddit and twitter content. One feature of this application is scheduling daily runs:



If you're interested in how this was created, and how to create your own scheduling tool with Electron and Vue (think for an alarm clock, a daily reminder, a message scheduler, etc.), then read on!

--

`npm install -g @vue/cli`

`vue create electron-vue-node-scheduler`

I like to use class-style component syntax, so to follow along with my code, select "typescript" and "Use class-style component syntax"

`cd electron-vue-node-scheduler`

`vue add electron-builder`

Now, let's build prototype a simple component that will act as our user interface. We can use Vue Bootstrap to speed up development.

Install with `vue add bootstrap-vue`.

Now, we can start up our app with `yarn electron:serve`.

Alright, now we can build the actual component.

--

You new boilerplated Electron/Vue app comes with a convenient HelloWorld.Vue component which we will be modifying to be our scheduler component.

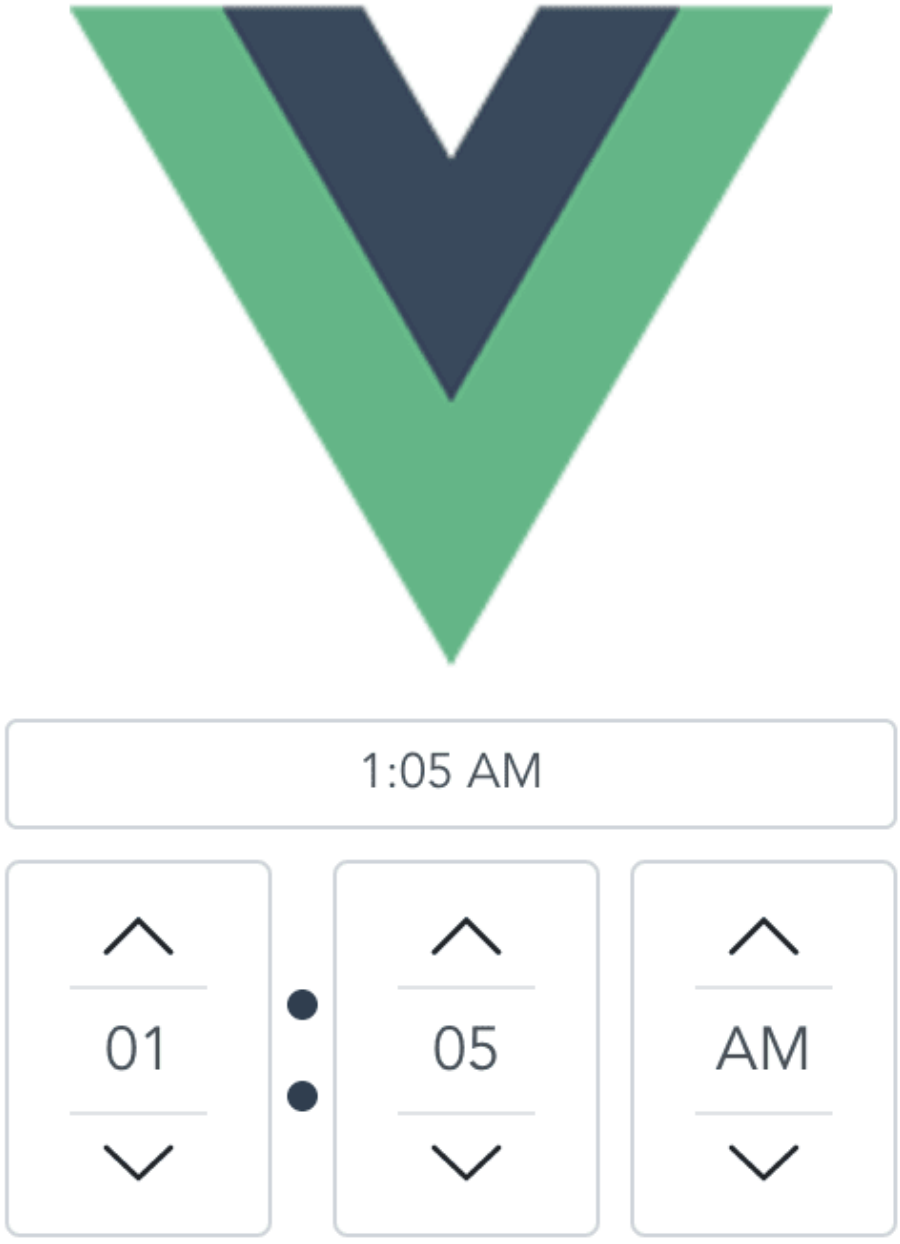
Go ahead and copy paste this code over everything that was in HelloWorld.Vue:

```
<template>
  <div>
    <b-time
      v-model="value"
      locale="en"
      @context="onContext"
    ></b-time>
  </div>
</template>

<script lang="ts">
import { Component, Prop, Vue } from 'vue-property-decorator';

@Component
export default class HelloWorld extends Vue {}
</script>
```

```
</style>
```

The b-time here is the bootstrap-vue time component.

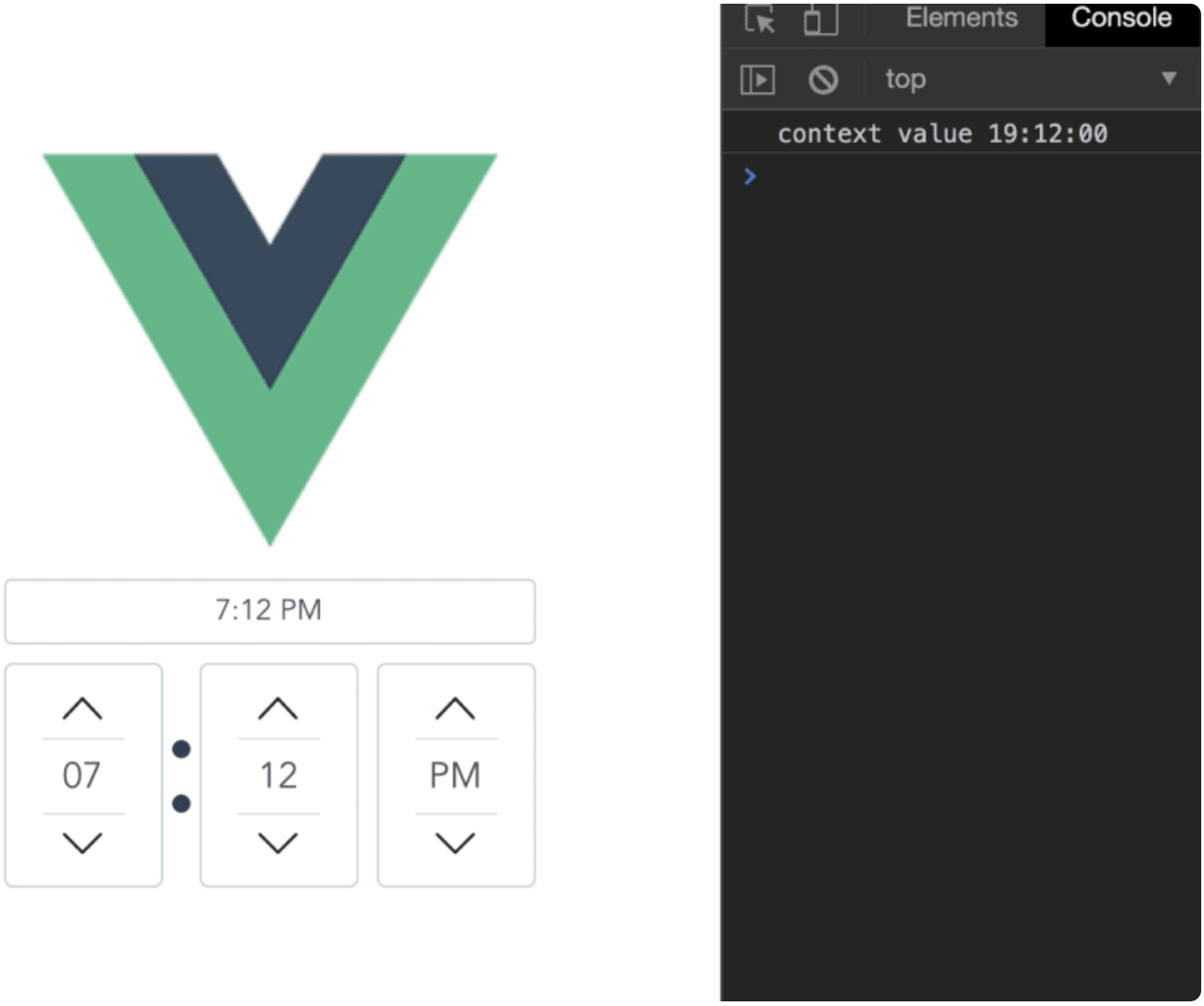
We now have set up an easy to use and good looking time selector component. Go ahead and play around with it:



Great, now let's get the actual time out of this so we can use it for our scheduler.

```
export default class HelloWorld extends Vue {
  value = '';

  onContext(context) {
    console.log('context value', context.value);
  }

  data() {
    return {
      value: this.value,
    };
  }
}
```

Now, we are keeping track of the value using Vue's data function and we will get an update through `onContext` whenever the time picker is changed by the user:



`yarn add node-schedule`

Then in your add a `import schedule from "node-schedule";`.

Then update your component like so:

```
export default class HelloWorld extends Vue {
  value = '';

  // eslint-disable-next-line @typescript-eslint/no-empty-function
  scheduleJob = schedule.scheduleJob('* * * * *', () => {});

  onContext(context) {
    // get hours and minutes off of something like '19:12:00'
    const hours = context.value.split(':')[0];
    const minutes = context.value.split(':')[1];

    this.scheduleJob.cancel();
    this.scheduleJob = schedule.scheduleJob(
      `${minutes || '00'} ${hours || '00'} * * *`,
      () => {
        console.log('job ran on schedule!');
      },
    );
  }

  data() {
    return {
      value: this.value,
    };
  }
}
```

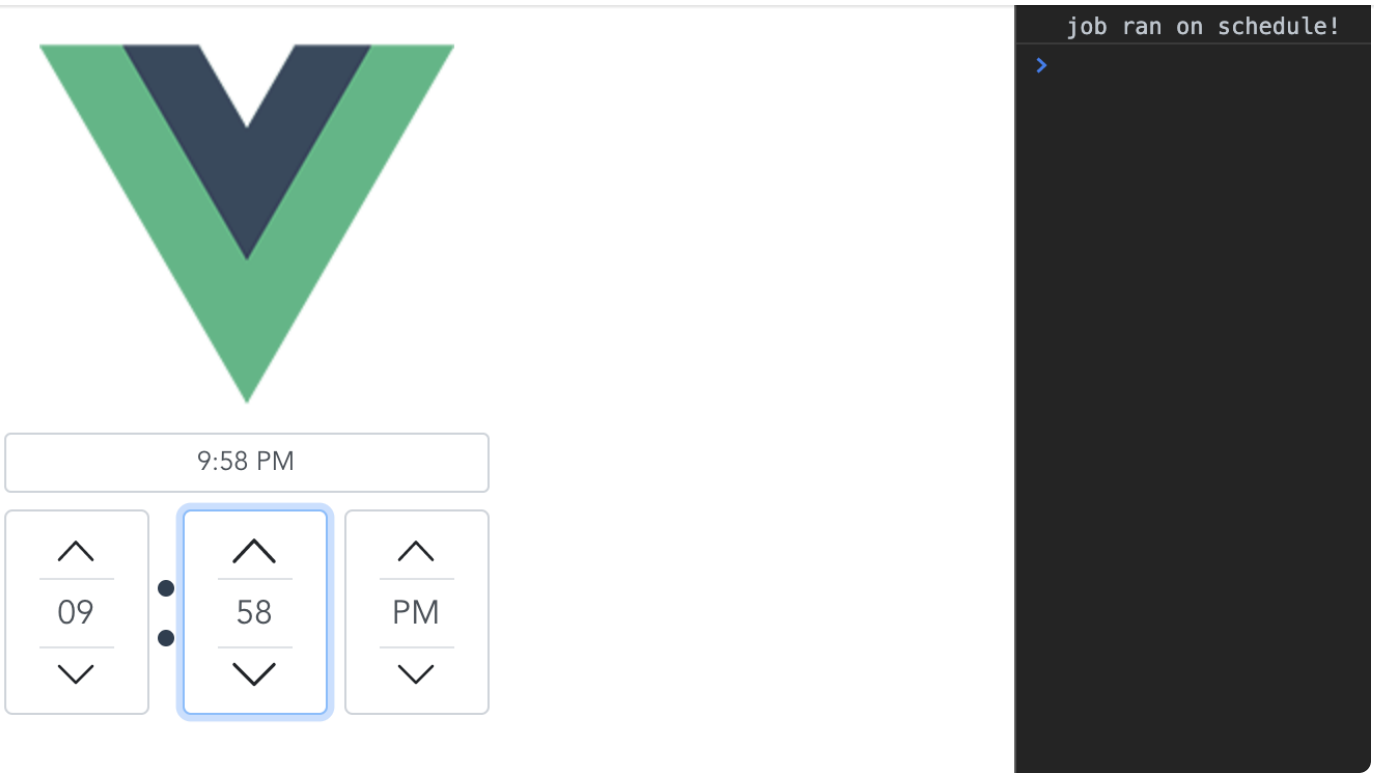
Okay, this is getting a bit more confusing. Let's break it apart:

`scheduleJob = schedule.scheduleJob('* * * * *', () => {});` This creates a local scheduleJob we can reference. We need to keep track of this, because any time the time is updated, we are going to cancel any previously scheduled jobs and create a new scheduled job.

```
*    *    *    *    *    *
┬    ┬    ┬    ┬    ┬    ┬
│    │    │    │    │    │
│    │    │    │    │    └ day of week (0 - 7) (0 or 7 is Sun)
│    │    │    │    └──── month (1 - 12)
│    │    │    └─────────── day of month (1 - 31)
│    │    └──────────────── hour (0 - 23)
│    └───────────────────── minute (0 - 59)
└────────────────────────── second (0 - 59, OPTIONAL)
```

Now, onContext (meaning whenever the user changes the time schedule component), we will get a string in the format of 'HH:MM:SS'. We want to break up this string to get hours, and minutes, which we do with the split function: `const hours = context.value.split(':')[0];`

Finally, we `cancel` the previous scheduled job, and create a new one at the time the user has now decided:



Great, that's it! You now have everything you need to build your own scheduler app! And how cool is this, since it's in electron, it's a native app that will run on Windows, Mac and Linux!

Next I would recommend looking into Electron's Tray functionality so that you can minimize the app to the tray instead of closing it, so your scheduled job will still run at the right time while the app is running in the background.

Here is the GitHub Repo that you can look at if you want to see the code I used in this post.

Top comments (0)